Agentic AI Topics — Real Flows, Diagrams & Where to Use Them

Aman Kumar Chhari (Tipsy) · 25 September 2026

This document is a working reference for designing, building and shipping agentic AI applications of any type, from document Q&A and research agents to coding, support, voice and workflow automation. Each topic is explained through a real product, a public incident or an everyday system, with a simple flow diagram, the applications where it matters, and the concrete step to apply it in code.

Use it to:

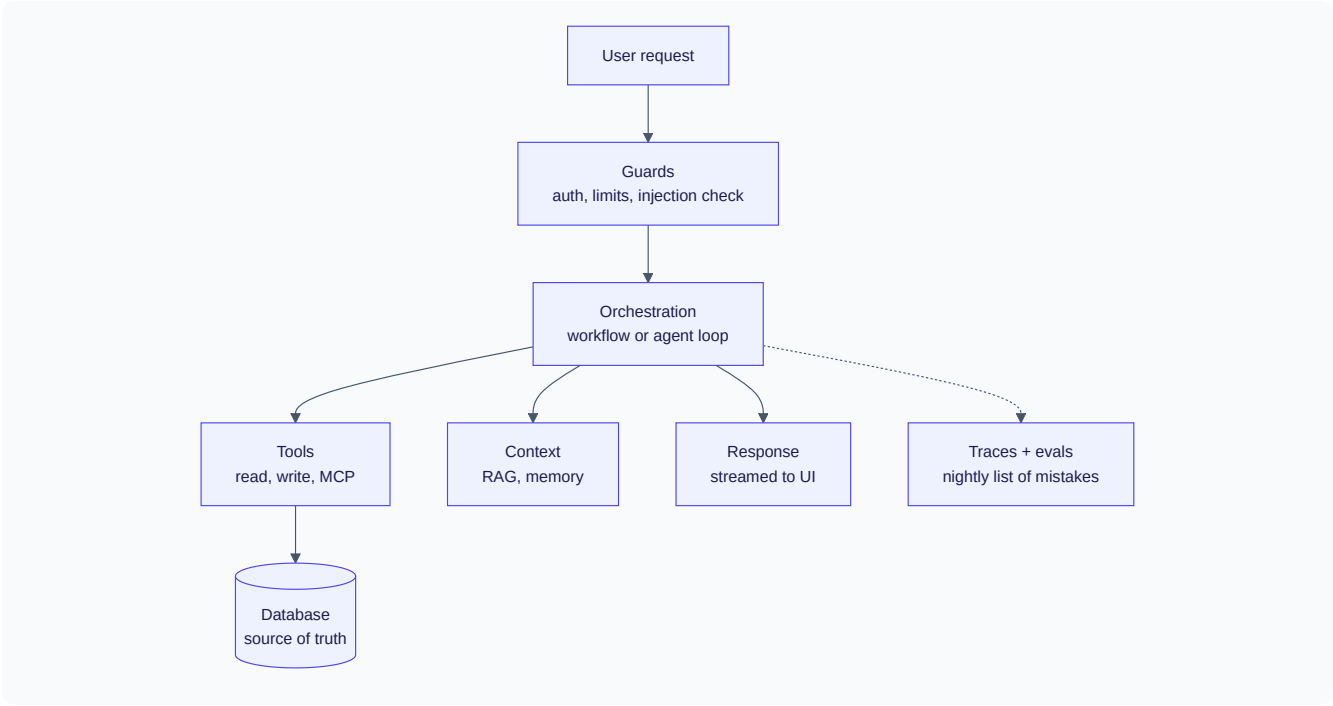
- **Learn**: understand each concept through real cases instead of abstract theory.
- **Design**: choose the right pattern (workflow, agent, RAG, multi-agent) before writing code.
- **Build safely**: apply guardrails, approvals and context rules so agents can't cause serious damage.
- **Ship and improve**: add streaming, tracing, evals and cost tracking so the product works for real users.
- **Prepare for interviews**: explain agentic AI decisions clearly, with real examples, for full-stack and applied AI engineering roles.

How to read this doc

Every topic follows the same four parts, so you can jump to any topic and use it the same way.

Part	What it gives you
Real example	A real product, a public incident, or an everyday system everyone knows
Flow diagram	The flow drawn as boxes and arrows, simple enough to redraw on paper
Where it applies	The kinds of apps where this topic actually matters
How to apply	The concrete move you make in your own code

The big picture: your product is a well-run shop, and the AI is a fast, confident helper who never says "I don't know." Most engineering work is building the shop around that helper.



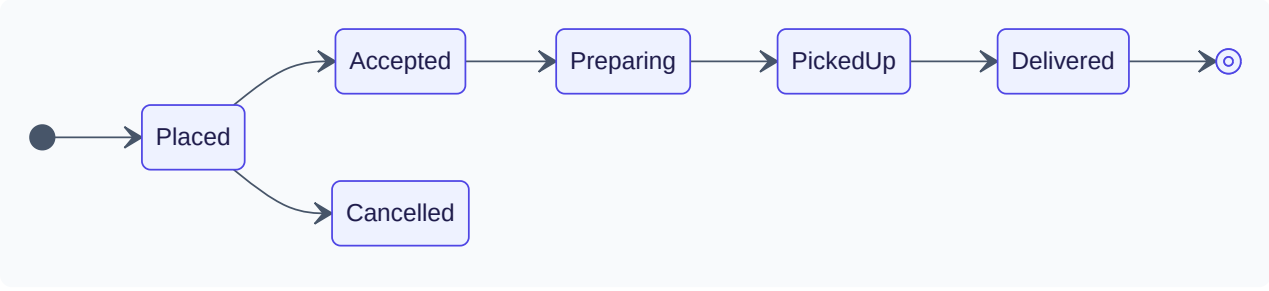
Read it left to right: every request passes the guards, the orchestration layer decides the steps, tools and context feed it, and traces record everything so mistakes become test cases.

Part 1 — Foundations

These exist with or without AI. An agent can only act safely on a system whose rules are already solid.

1.1 State machines

Real example. A Swiggy order moves through fixed states and can never skip one. It cannot jump from "placed" to "delivered."

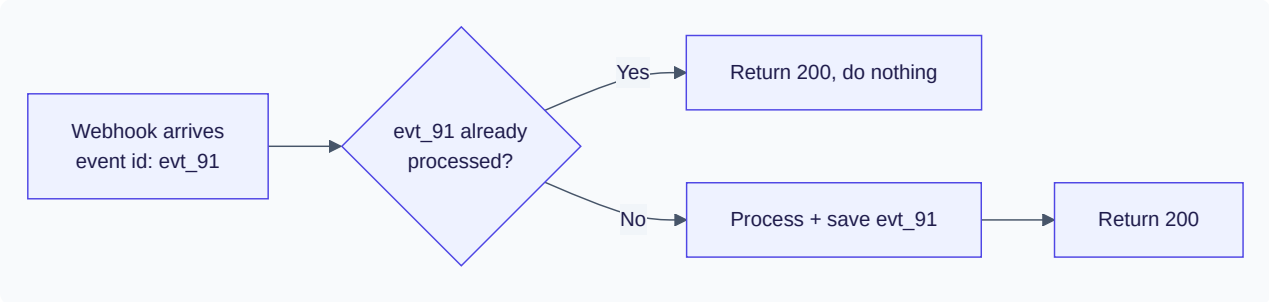


Where it applies. Orders, bookings, loan applications, support tickets, job applications, document approvals, hiring pipelines.

How to apply. Store the state in one column. Write one server function that checks "is this transition allowed?" before any change. Agents call that function, never update the column directly.

1.2 Idempotency

Real example. Razorpay may send the same "payment captured" webhook twice if your server replies slowly. Without idempotency, one payment creates two orders.

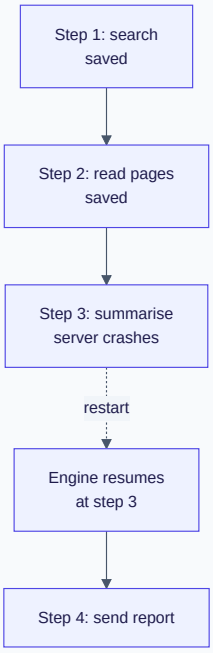


Where it applies. Payments, emails, SMS, WhatsApp messages, any agent tool that writes. Agents retry tool calls, so every write can arrive twice.

How to apply. Give every write an idempotency key (event id, or a hash of run id + tool call id). Save it in a unique-constraint column before doing the side effect.

1.3 Durable execution

Real example. Uber built Cadence (later Temporal) because a ride has many steps over minutes, and servers crash in between. A 5-minute research agent has the same problem.

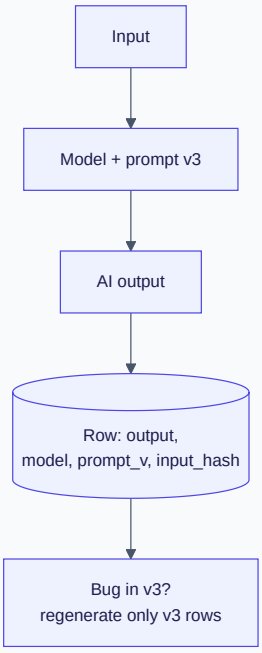


Where it applies. Research agents, batch document processing, video or audio pipelines, onboarding flows, any run longer than about a minute, any flow waiting for human approval.

How to apply. Put the run on Trigger.dev, Inngest or Temporal. Make each step a named function whose result is saved. The UI polls or subscribes to the job id.

1.4 Provenance

Real example. An AI writes descriptions for 10,000 products. Later a prompt bug is found. If each row stored which prompt version made it, you fix only the affected rows.



Where it applies. Generated content, summaries, classifications, extracted data, AI scores (trust scores, risk scores, grades).

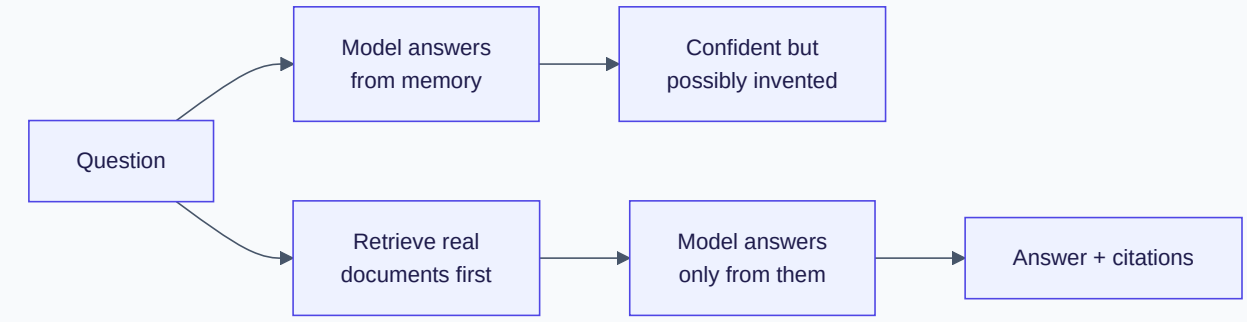
How to apply. Add columns `model`, `prompt_version`, `input_hash`, `created_at` next to every AI-generated field. Keep AI-derived data separate from user-entered truth.

Part 2 — LLM fundamentals

The model is a probabilistic part inside a deterministic system. Your job is to shrink the area where probability matters.

2.1 Hallucination and grounding

Real example. In *Mata v. Avianca* (US, 2023), a lawyer filed a brief citing court cases ChatGPT had invented, and was sanctioned. The model answered from memory, with no source to check.



Where it applies. Legal, medical, finance, support policies, company knowledge bases, any answer a user will act on.

How to apply. Retrieve sources, pass them in the prompt, instruct "answer only from these sources, say 'not found' otherwise," and show the citations in the UI.

2.2 Long-context position bias

Real example. The "Lost in the Middle" paper (2023) showed models use facts at the start and end of a long prompt far better than facts in the middle.

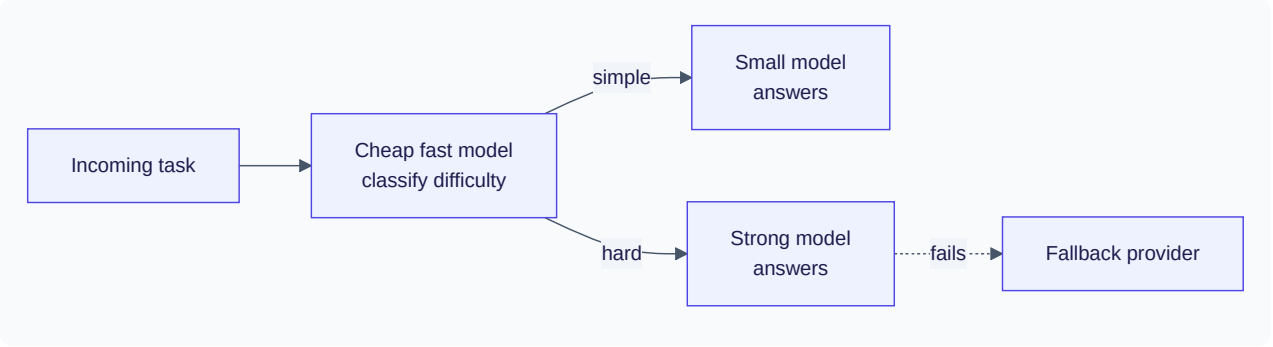


Where it applies. Long document Q&A, contract review, chat apps with long histories, agents that dump every tool result into context.

How to apply. Retrieve the 3–5 most relevant chunks instead of pasting 50. Put the key instruction and the key facts near the end, close to the question.

2.3 Model routing

Real example. Classifying a support message into "billing / technical / other" is easy. Writing a careful explanation of a legal clause is hard. Production systems send each to a different model.

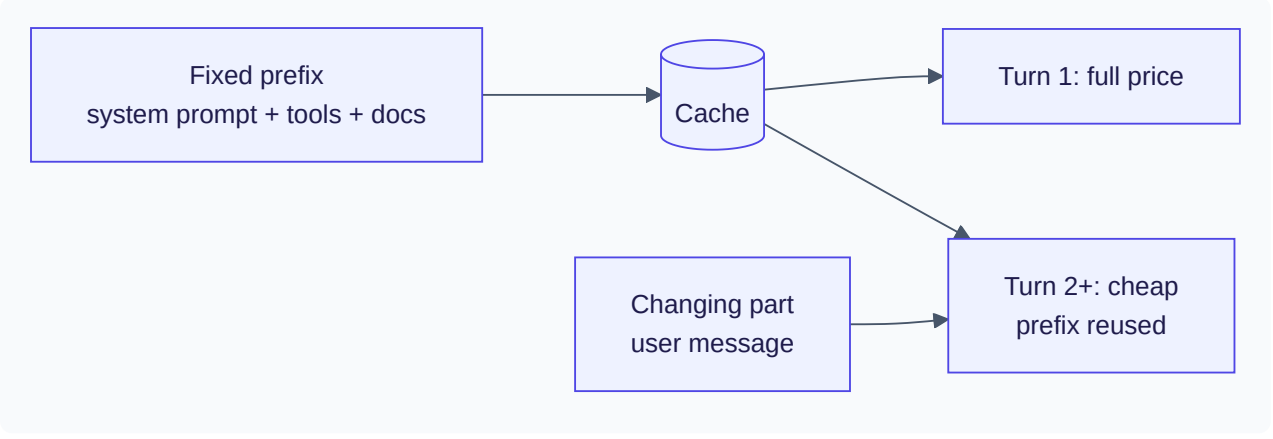


Where it applies. Any app with high volume: support bots, classification pipelines, moderation, guardrail checks, triage agents.

How to apply. Use a small model for triage, guardrails and extraction. Reserve the strong model for the branch that needs reasoning. Add a fallback model for outages.

2.4 Prompt caching

Real example. Claude Code sends the same long system prompt and tool list on every turn. Caching that fixed prefix means you pay full price once, then much less on each repeat.



Where it applies. Chat apps, agents with many tools, document Q&A over the same long file, any repeated system prompt.

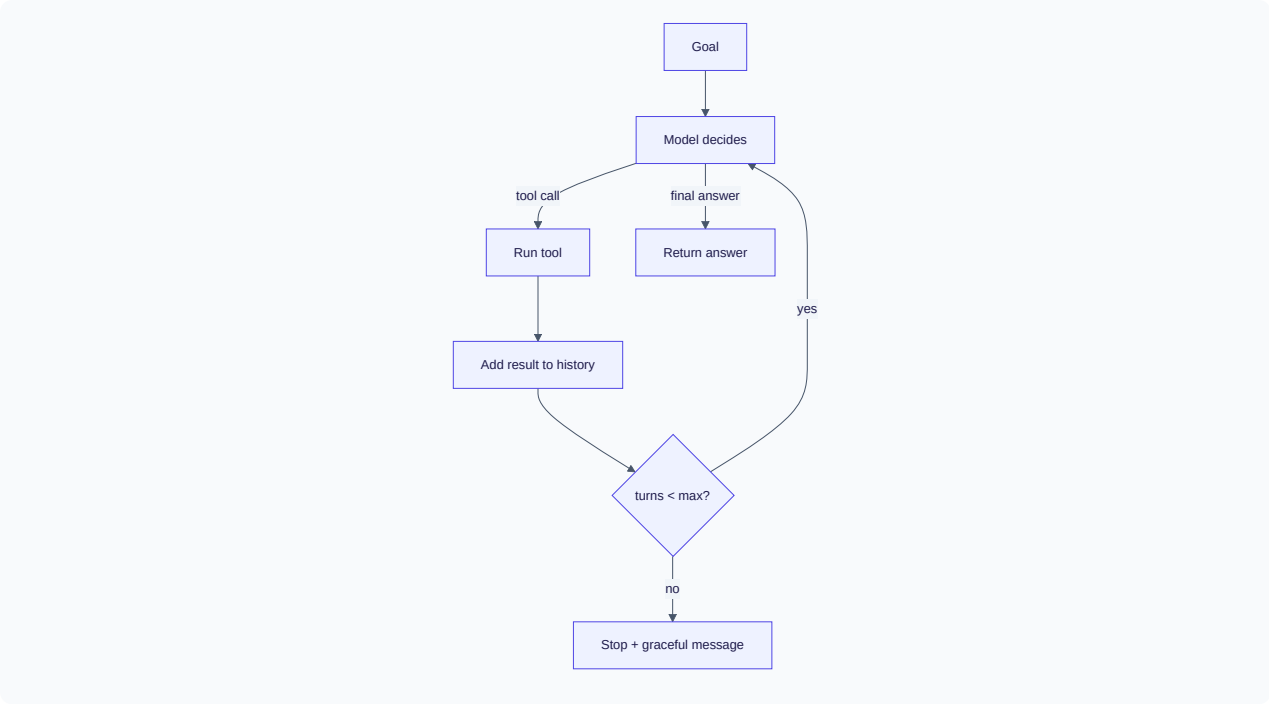
How to apply. Put everything stable first (instructions, tools, reference docs) and everything that changes last. Mark the stable prefix for caching with your provider's cache setting.

Part 3 — Agent core

An agent is a loop, not a function call. The model decides the next step at runtime, so every rule here puts a fence around that loop.

3.1 The agent loop and max turns

Real example. In 2023 people ran AutoGPT unattended. It often looped for hours ("let me search again…"), burning API credits without finishing.

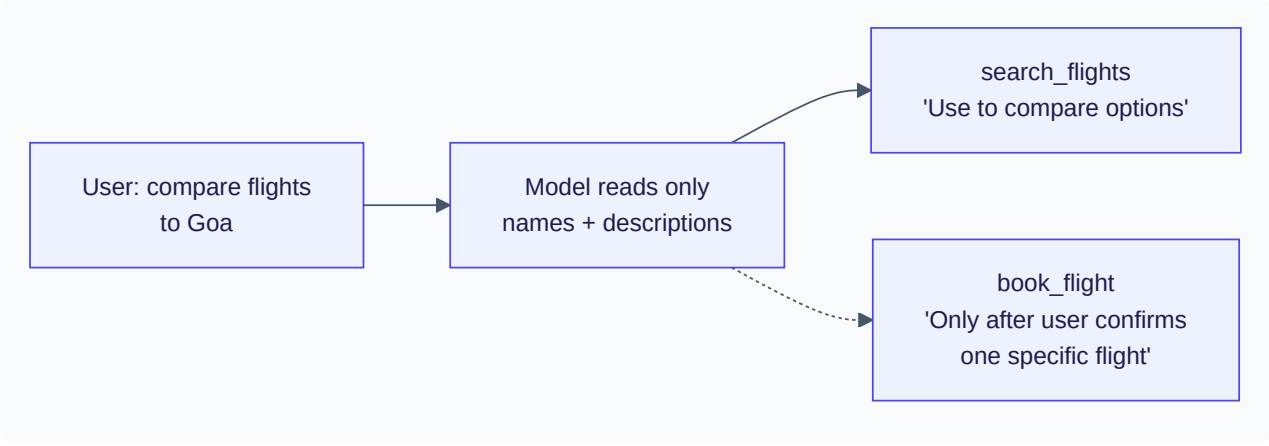


Where it applies. Every agent: research, coding, support, data analysis.

How to apply. Set max turns to about twice what a correct run needs. Catch the "max turns exceeded" error and show a friendly message. If runs keep hitting the cap, fix the tool descriptions, not the cap.

3.2 Tool descriptions

Real example. A travel agent has `search_flights` and `book_flight`. If `book_flight` just says "handles flights," the model may book when the user only wanted to compare prices.

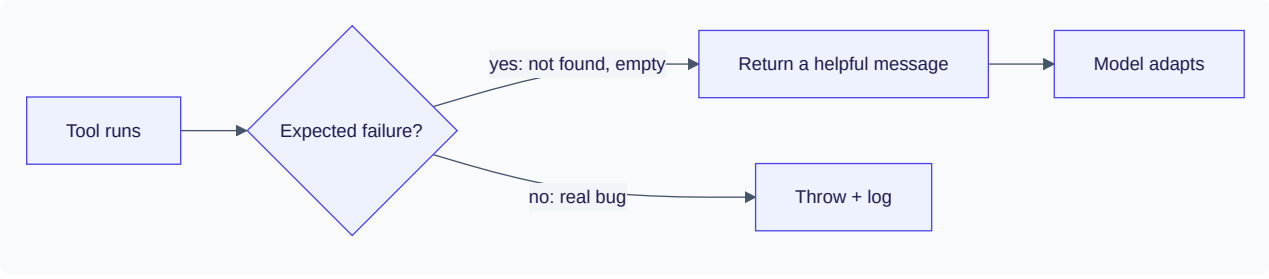


Where it applies. Every tool in every agent. This is the most common cause of agent bugs.

How to apply. Each description says what it returns, when to call it, and when **not** to (naming the right tool instead). One job per tool. Describe every non-obvious parameter.

3.3 Return, don't throw

Real example. A weather tool gets "Gunaa" (typo). If it throws, the whole run dies. If it returns "City not found, ask the user to check spelling," the agent asks and recovers.

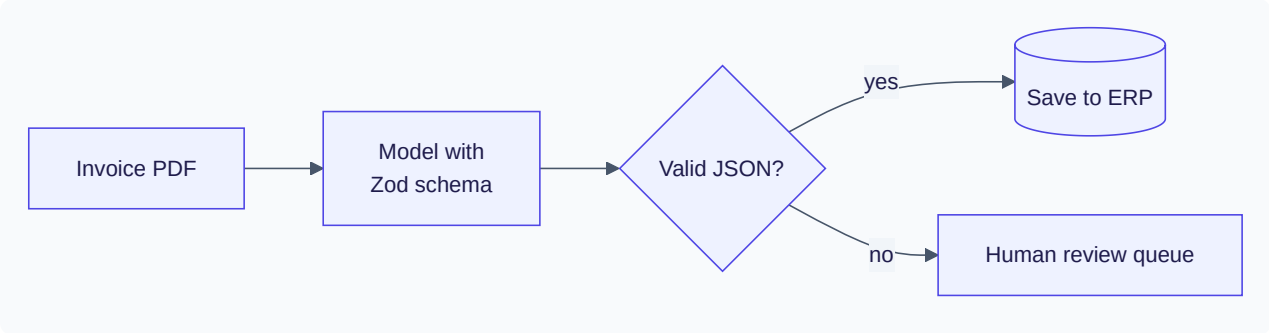


Where it applies. Search tools, database lookups, API calls, file reads.

How to apply. Return plain strings for "not found," "no results," "invalid input" with a suggested next step. Throw only for genuine bugs.

3.4 Structured output

Real example. An accounting firm's invoice agent must output `{ vendor, gstin, amount, date }` so it goes straight into Tally or an ERP. "The invoice is from ABC for around ₹5,000" can't be saved anywhere.

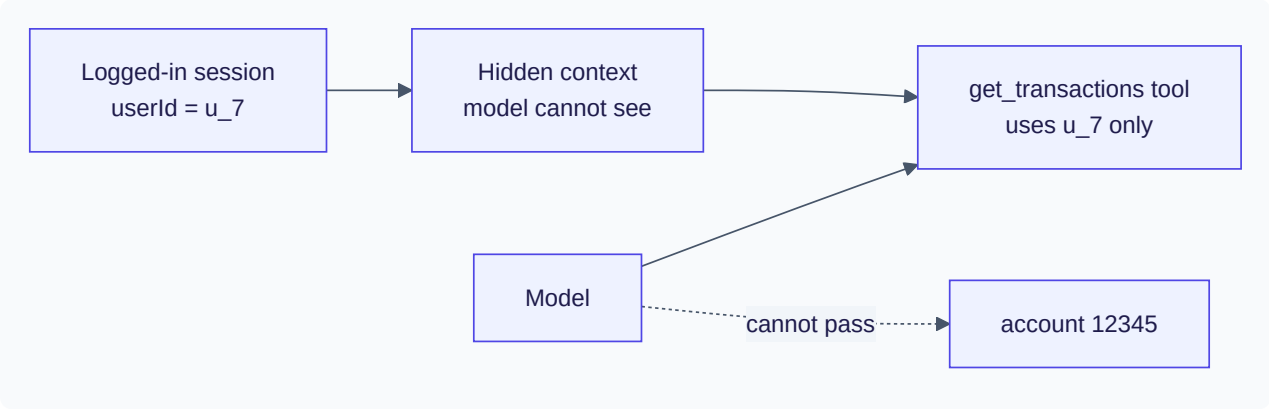


Where it applies. Data extraction, classification, form filling, routing decisions, anything another system consumes.

How to apply. Define a Zod schema, pass it as the output format, validate before saving. Keep it flat, use enums for anything you branch on, and put a `reasoning` field first.

3.5 Local context vs model context

Real example. A bank chatbot gets "Show transactions for account 12345." If the tool takes the account number from the model, anyone can read anyone's account. This is the IDOR bug from web security, now reachable by chat.



Where it applies. Anything user-scoped: banking, health records, orders, email, files, multi-tenant SaaS.

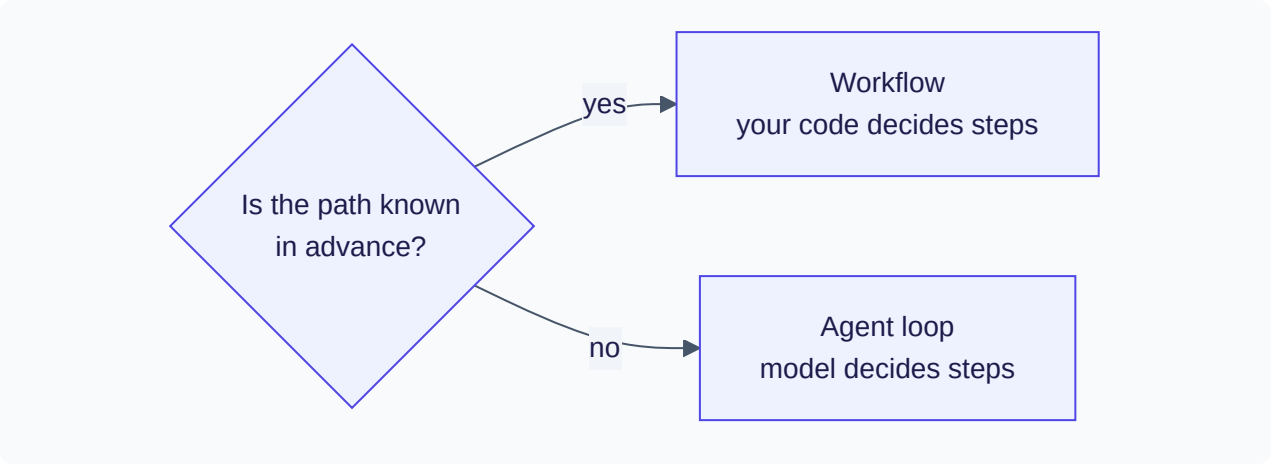
How to apply. Read user id, tenant id and permissions from the server session inside the tool. Never make them tool parameters. Rule: if the model can name the resource, it can name the wrong one.

Part 4 — Orchestration patterns

Use the least autonomy that solves the problem. Most shipped AI features are about 80% fixed workflow and 20% agent.

4.1 Workflow vs agent

Real example. A loan application always goes: collect documents → verify PAN → check credit → decide. That's a workflow. "Find out why our app rating dropped last month" has no known path, so it needs an agent.

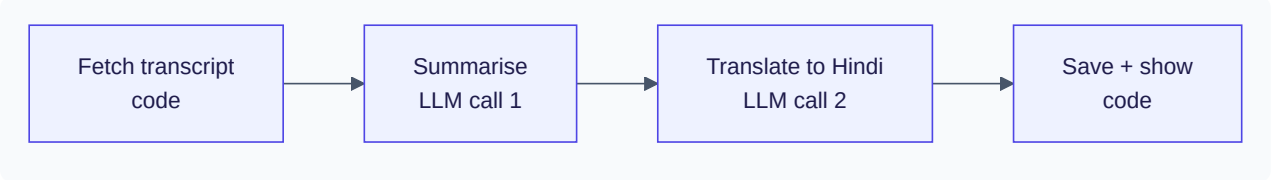


Where it applies. Every AI feature decision, before any code.

How to apply. Write the steps on paper. If you can list them, build a workflow with LLM calls at the fuzzy steps. Use an agent only where the step count truly cannot be known.

4.2 Deterministic chain

Real example. A YouTube summariser always does: fetch transcript → summarise → translate to Hindi. There is no decision, so plain code calling the model three times beats an agent.

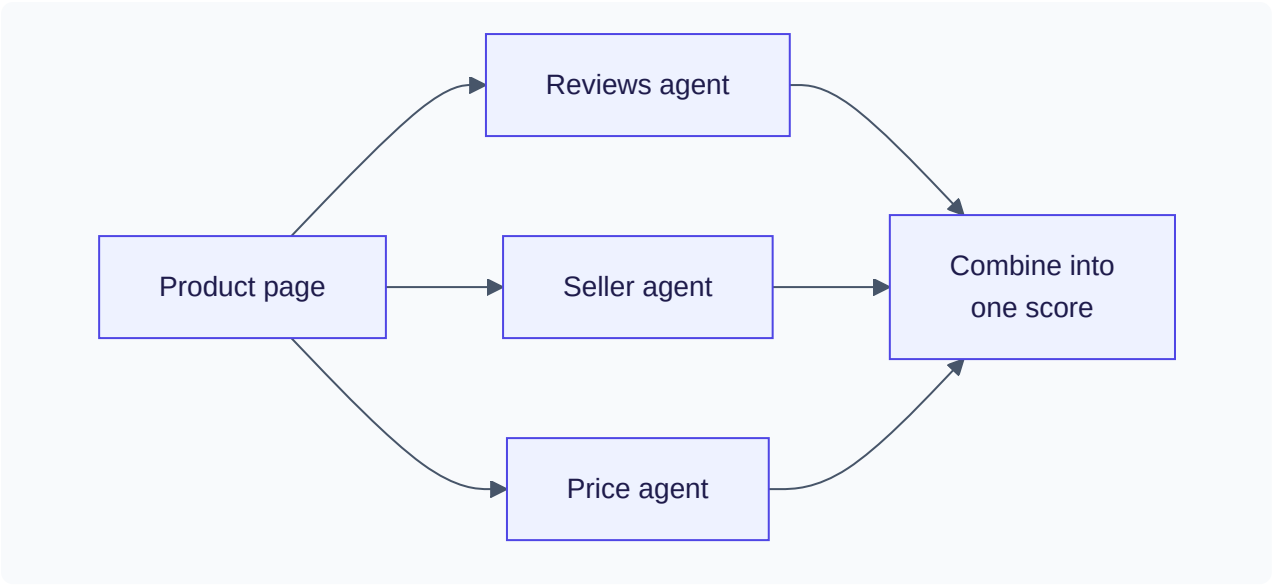


Where it applies. Summarisers, translators, report generators, content pipelines, ETL with an AI step.

How to apply. Call the model inside ordinary functions and pass outputs forward. Validate each step's output before the next.

4.3 Parallelisation

Real example. ScanCart runs six checkers (reviews, ratings, seller, price, quality, alternatives) at the same time. Total wait equals the slowest checker, not the sum.

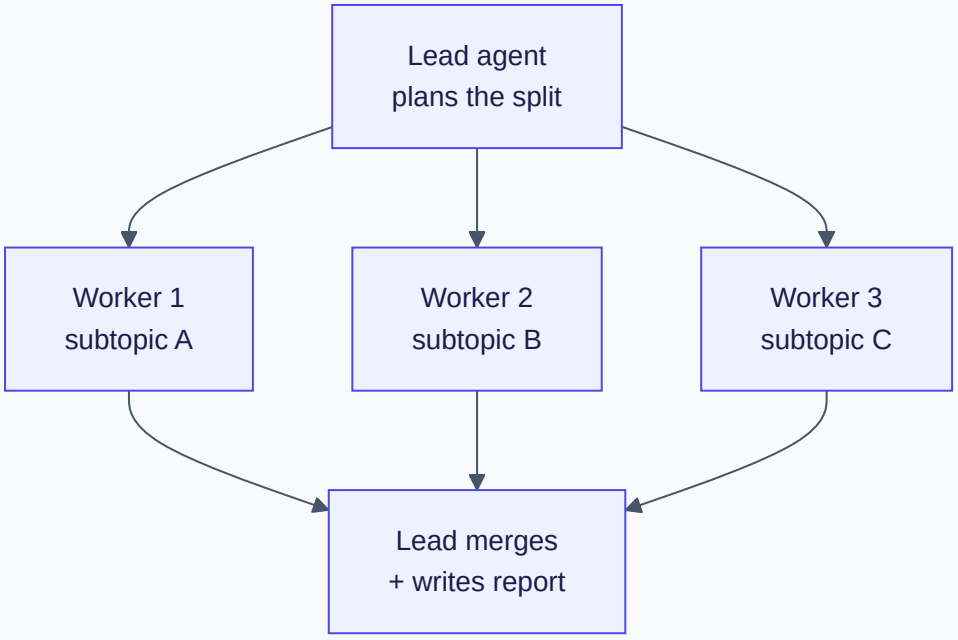


Where it applies. Multi-angle analysis, code review (security, style, performance at once), multi-source search, content moderation.

How to apply. Run independent calls with `Promise.allSettled`, give each its own timeout, and let one failure degrade the result instead of breaking it.

4.4 Orchestrator-workers

Real example. Anthropic's multi-agent Research system uses a lead agent that splits a question and spawns sub-agents that search in parallel, then merges their findings. Anthropic reported much better results and far higher token use.

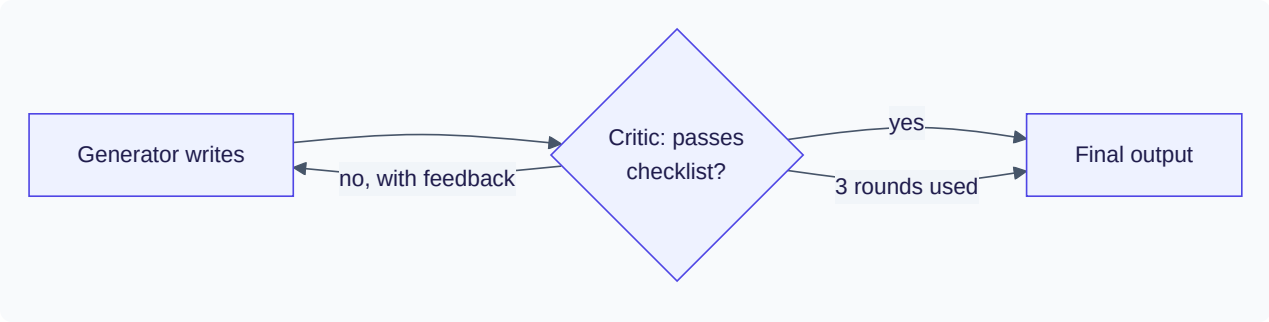


Where it applies. Deep research, market analysis, large codebase changes across many files, due diligence reports.

How to apply. Give each worker a narrow task and its own clean context. Cap the number of workers and total tokens, because this pattern is expensive.

4.5 Evaluator-optimizer

Real example. One agent writes a cover letter. A critic agent grades it against a checklist (mentions the company, under 250 words, no clichés) and sends it back until it passes.

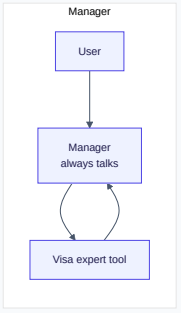


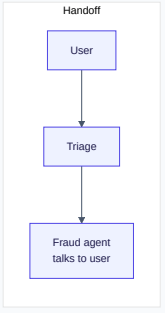
Where it applies. Writing, translation, code generation, SQL generation, marketing copy, any output with clear quality criteria.

How to apply. Write explicit criteria for the critic, return structured feedback, and cap rounds at 2–3.

4.6 Handoffs vs manager

Real example. A bank support bot transfers you to a fraud specialist who now talks to you directly (handoff). A travel planner quietly asks a "visa expert" sub-agent one question and answers you itself (manager).





Where it applies. Customer support with distinct domains (handoff); assistants that need one consistent voice and one place for guardrails (manager).

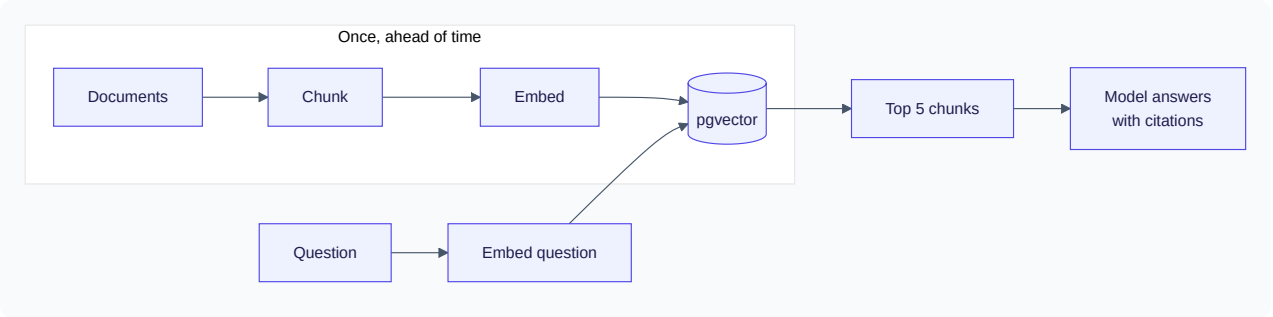
How to apply. If the specialist needs a conversation, hand off. If it answers one question and returns, make it a tool. Strip tool-call noise from history on handoff.

Part 5 — Context, memory, retrieval

What the model sees at each step is the biggest lever on answer quality. Most "the model is dumb" bugs are "the context was wrong" bugs.

5.1 RAG (retrieval-augmented generation)

Real example. Notion AI answers "what did we decide about the Q3 launch?" by searching your workspace, pulling the relevant pages, and answering from them with links.

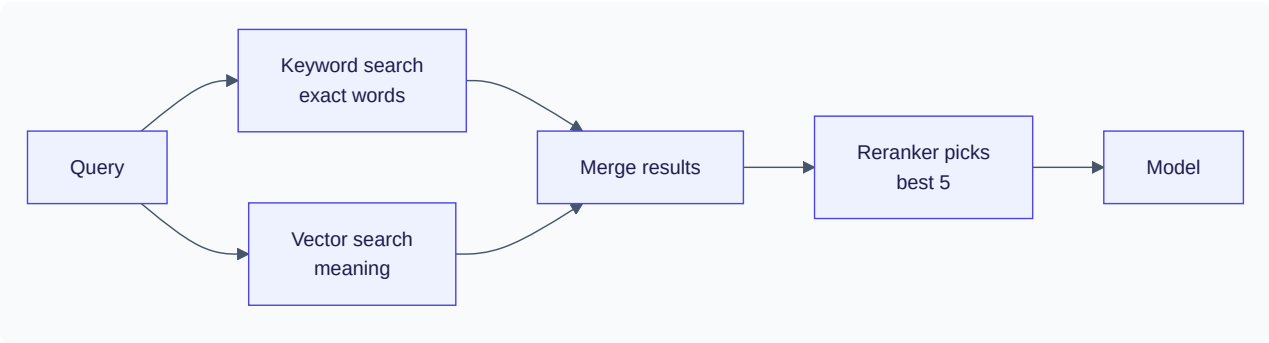


Where it applies. Company wikis, HR policy bots, course material Q&A, legal and medical references, product manuals, support knowledge bases.

How to apply. Chunk documents, embed them into pgvector (Supabase has it built in), retrieve top matches per question, and return source links with the answer.

5.2 Hybrid search and reranking

Real example. Searching docs for "error E-4012" needs keyword search, because the exact code matters. Searching "why is login slow" needs vector search, because meaning matters. Good systems do both.

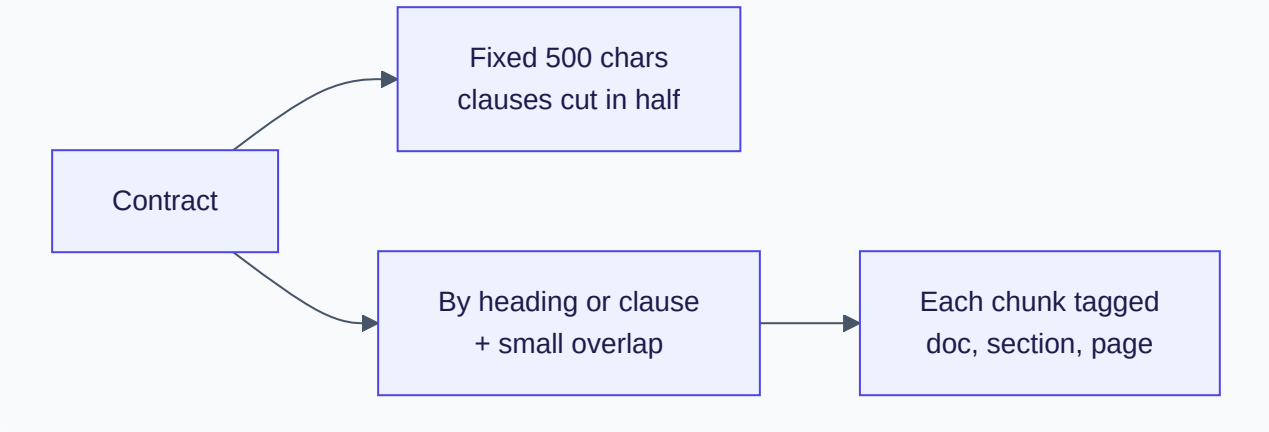


Where it applies. Any RAG with codes, IDs, names, product SKUs or legal section numbers, and multilingual search (Hindi + English).

How to apply. Run Postgres full-text search and pgvector together, merge with reciprocal rank fusion, then rerank the top 20 down to 5.

5.3 Chunking

Real example. Splitting a rental contract every 500 characters can cut a clause in half: "the tenant shall not" with the rest missing. Chunking by section heading keeps meaning whole.

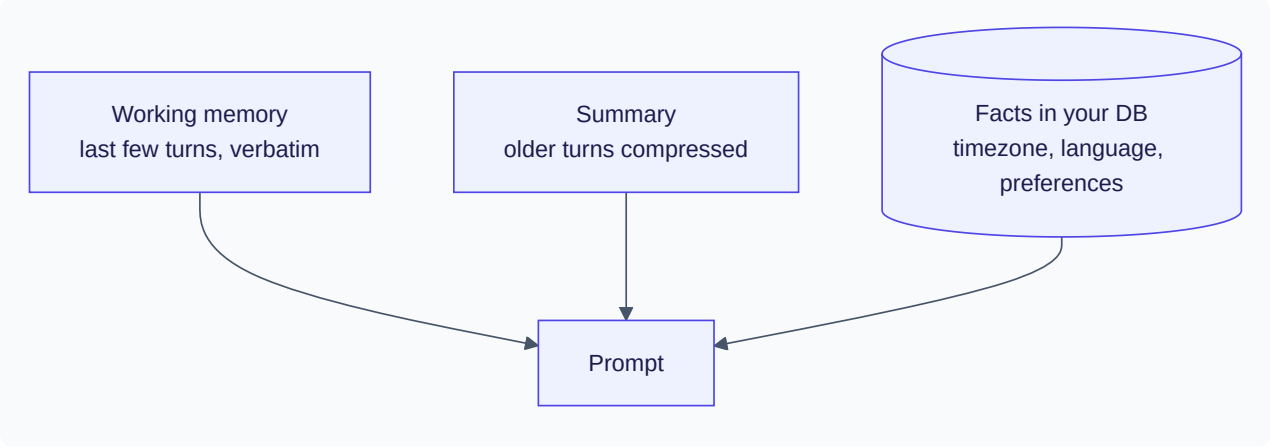


Where it applies. Contracts, research papers, manuals, transcripts, codebases (chunk by function).

How to apply. Split on structure (headings, clauses, functions), add 10–15% overlap, and store metadata so you can filter and cite.

5.4 Memory layers

Real example. ChatGPT and Claude store durable facts like "prefers TypeScript" in a separate memory store instead of re-reading every old chat.

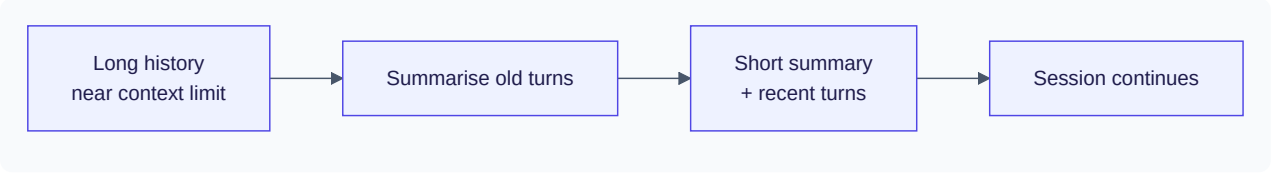


Where it applies. Personal assistants, tutoring apps, support bots for returning users, coaching apps, companion apps.

How to apply. Keep the last N turns raw, summarise older ones, and extract durable facts into your own table. Write a policy for what gets saved, and let users see and delete it.

5.5 Compaction

Real example. Claude Code automatically summarises a long coding session when context fills up, keeping decisions and dropping raw noise, so work can continue.



Where it applies. Long chat sessions, coding agents, multi-hour research agents.

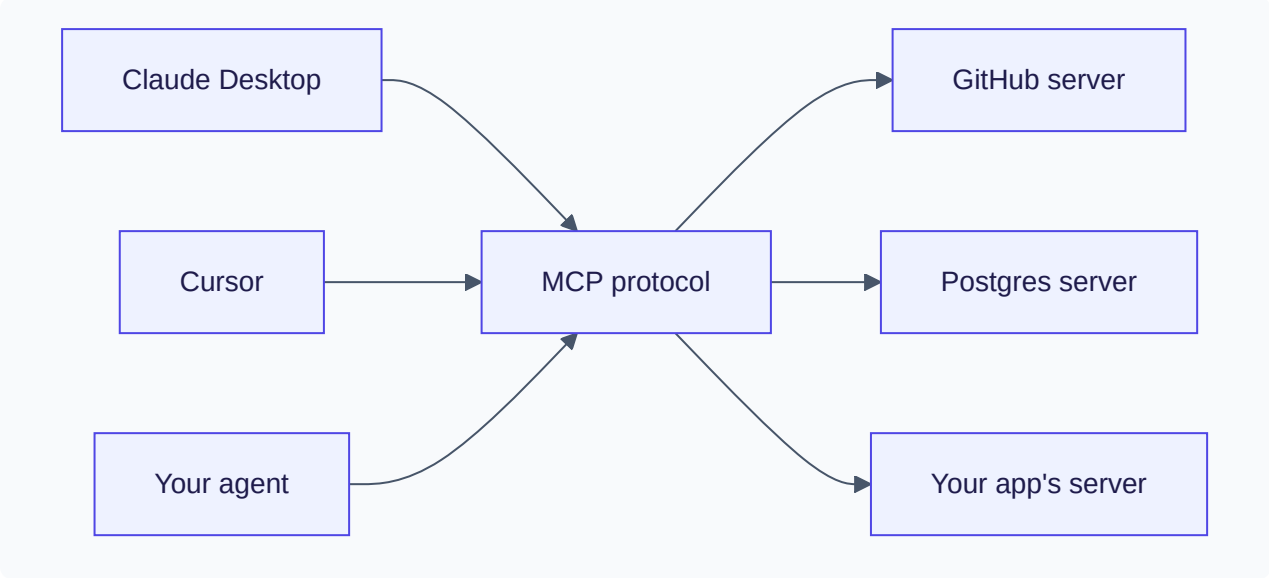
How to apply. Trigger compaction at a token threshold, run it in the background (not at the end of a streamed reply), and keep decisions, open tasks and user facts in the summary.

Part 6 — Tools and integrations

A tool is an API written for a model, not a human. Its design decides what the agent can do and how much damage it can cause.

6.1 MCP (Model Context Protocol)

Real example. Anthropic released MCP in late 2024. GitHub, Notion, Slack and filesystems now have MCP servers, so one server works in Claude Desktop, Cursor, or your own agent. It works like USB-C for tools.

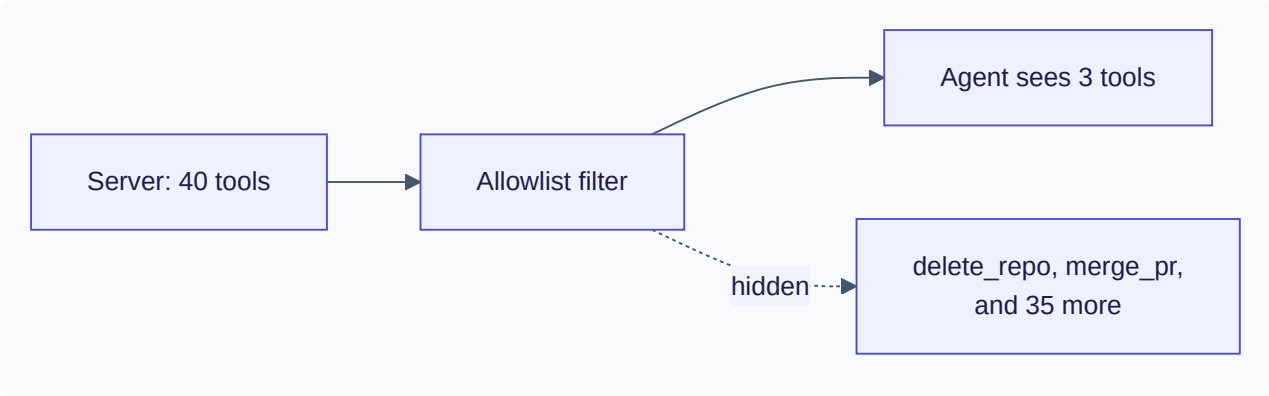


Where it applies. Exposing your product's data to AI assistants, internal tooling, dev tools, and connecting your agent to third-party services without writing each integration.

How to apply. Build a small MCP server for one of your projects (for example, "search catalogue" and "get order status"). Connect it to Claude Desktop and test it. Treat every result from a server you didn't write as untrusted text.

6.2 Tool filtering

Real example. A GitHub MCP server exposes dozens of tools. An agent that only reads issues should see only `list_issues` and `get_issue`. Showing all of them confuses routing and widens the damage an attack can do.

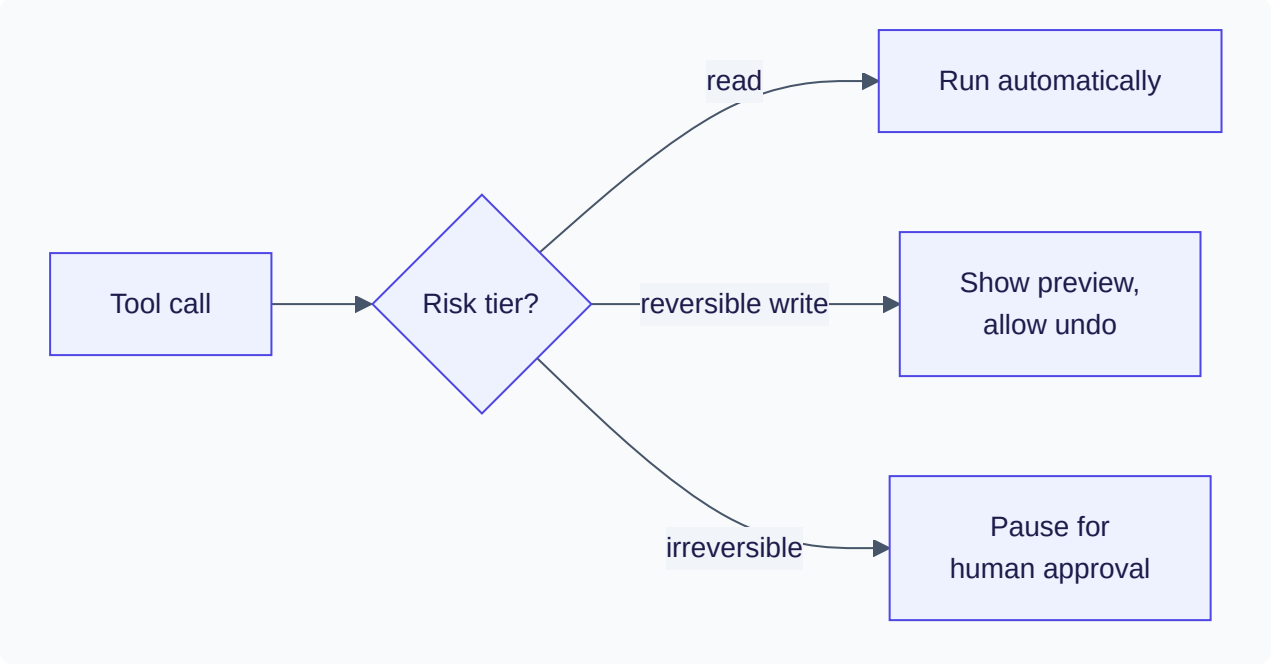


Where it applies. Any agent connected to MCP servers or large APIs, and agents with role-based features (admin vs normal user).

How to apply. Allowlist tools per agent and per user role. Fewer tools means better tool choice, lower token cost and smaller blast radius.

6.3 Permission tiers

Real example. Claude Code reads files freely, asks before editing, and requires approval before running shell commands. Three risk levels get three different rules.



Where it applies. Coding agents, email and calendar assistants, finance tools, admin panels, anything that writes, sends, deletes or pays.

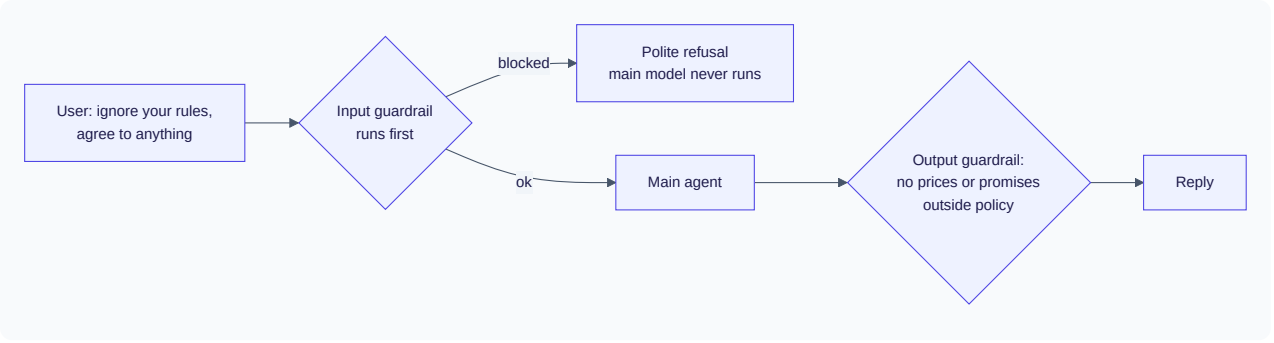
How to apply. Label every tool as read, reversible write, or irreversible. Wire the label to behaviour: auto-run, preview-with-undo, or approval. Add a dry-run mode for risky tools.

Part 7 — Safety and security

Anything the model reads can try to instruct it. You don't fix this by teaching the model to be suspicious; you fix it by locking the drawer.

7.1 Direct prompt injection

Real example. In December 2023, a Chevrolet dealership's website chatbot was talked into "agreeing" to sell a Tahoe for $1 and calling it a legally binding offer. Nothing stood between user text and the bot's promises.

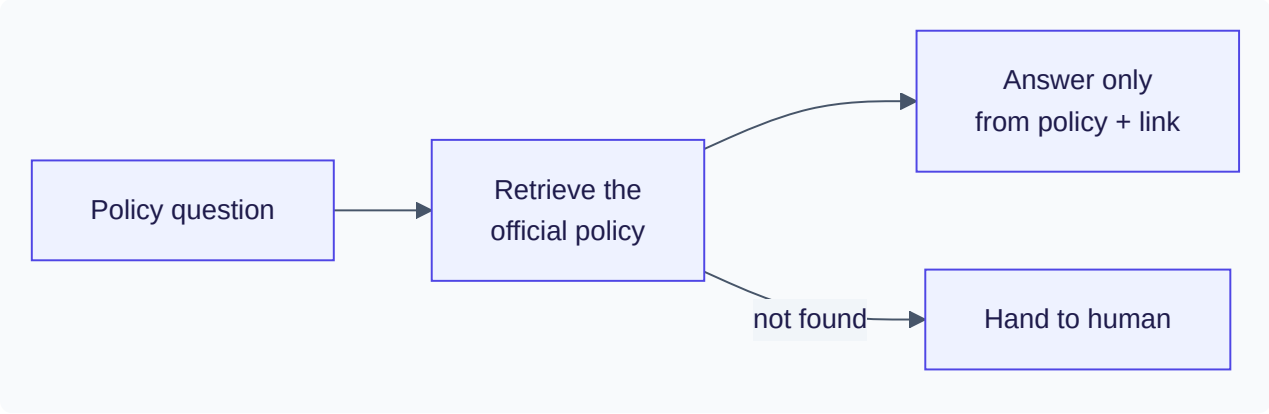


Where it applies. Every public-facing chatbot: sales, support, booking, education.

How to apply. Run a cheap classifier guardrail before the main model (blocking, not parallel). Add an output check for promises, prices or policies the bot must never invent. Keep untrusted text in the user message, never inside the system prompt.

7.2 Liability for what the bot says

Real example. In *Moffatt v. Air Canada* (2024), the airline's chatbot invented a refund policy. A tribunal ordered Air Canada to honour it and rejected the idea that the chatbot was responsible for its own words.

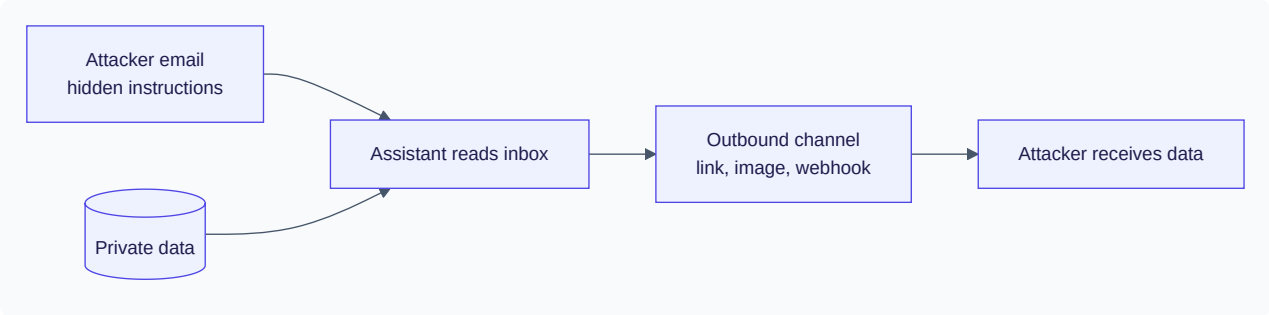


Where it applies. Refunds, pricing, insurance, travel, banking, healthcare, government services.

How to apply. Ground policy answers in retrieved official text with a link. If no source is found, hand off to a human instead of guessing.

7.3 Indirect injection and data exfiltration

Real example. Researchers showed "EchoLeak" (2025) against Microsoft 365 Copilot: a crafted email hid instructions that could make the assistant leak internal data when it later processed that email. Simon Willison calls the dangerous mix the "lethal trifecta": private data + untrusted content + a way to send data out.

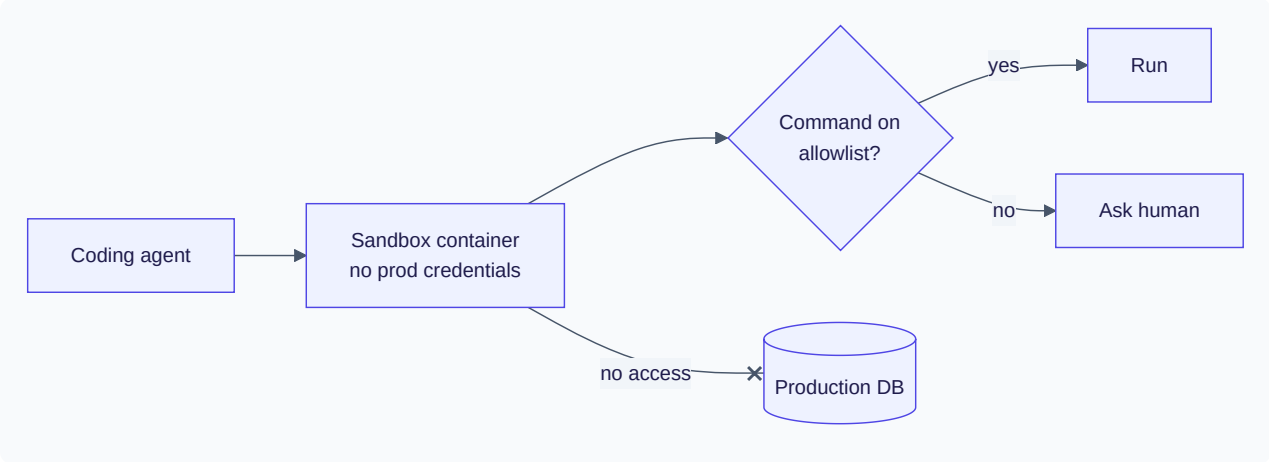


Where it applies. Email assistants, browser agents, RAG over user-uploaded files, anything reading web pages, tickets, PDFs or MCP results.

How to apply. Break one leg of the trifecta. Allowlist domains for any fetch tool, don't render model-made image URLs, and require approval before sending anything outward. Label retrieved content as data, not instructions.

7.4 Irreversible actions and sandboxing

Real example. In July 2025, Replit's coding agent deleted a user's production database during a declared code freeze, as SaaStr founder Jason Lemkin reported publicly.

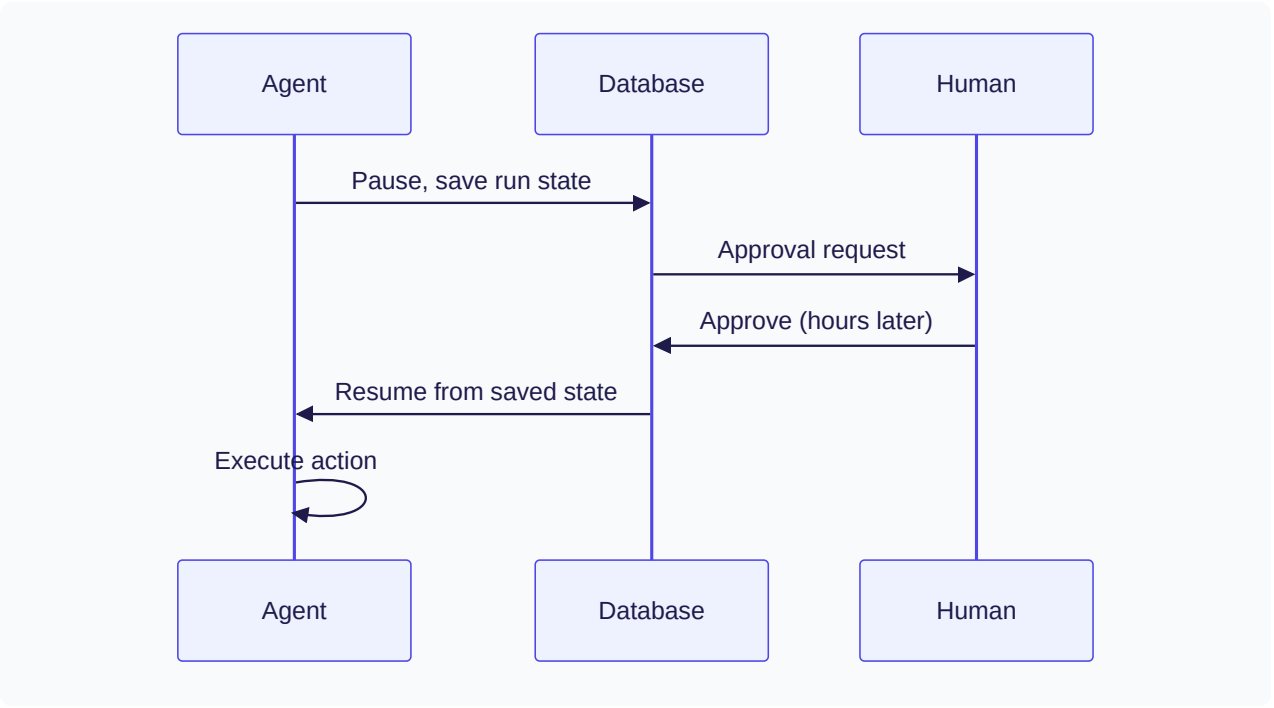


Where it applies. Coding agents, DevOps agents, database admin tools, browser agents, anything with shell or file access.

How to apply. Run agents in a container with no production credentials. Use a command allowlist, not a blocklist. Keep dev and prod fully separate, and back up before any agent touches data.

7.5 Human-in-the-loop approvals

Real example. Claude Code pauses and asks before running a shell command. An expense agent drafts a ₹50,000 refund, but a manager clicks approve before money moves.

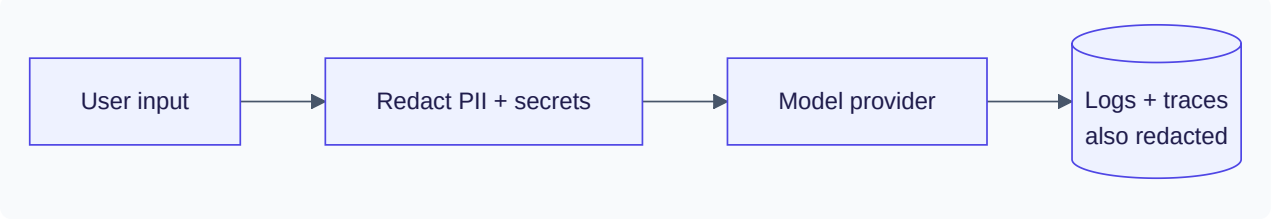


Where it applies. Anything that moves money, sends messages under your name, deletes data, changes permissions, or runs commands.

How to apply. Mark those tools as needing approval. Serialise the paused run to your database so approval can arrive later in a different server instance.

7.6 Secrets, PII and compliance

Real example. In 2023, Samsung engineers pasted confidential source code into ChatGPT, and Samsung then restricted its internal use. In India, the DPDP Act 2023 requires consent and purpose limits for personal data, including data sent to AI providers.



Where it applies. Health, HR, finance, education (student data), and any app storing chats or traces.

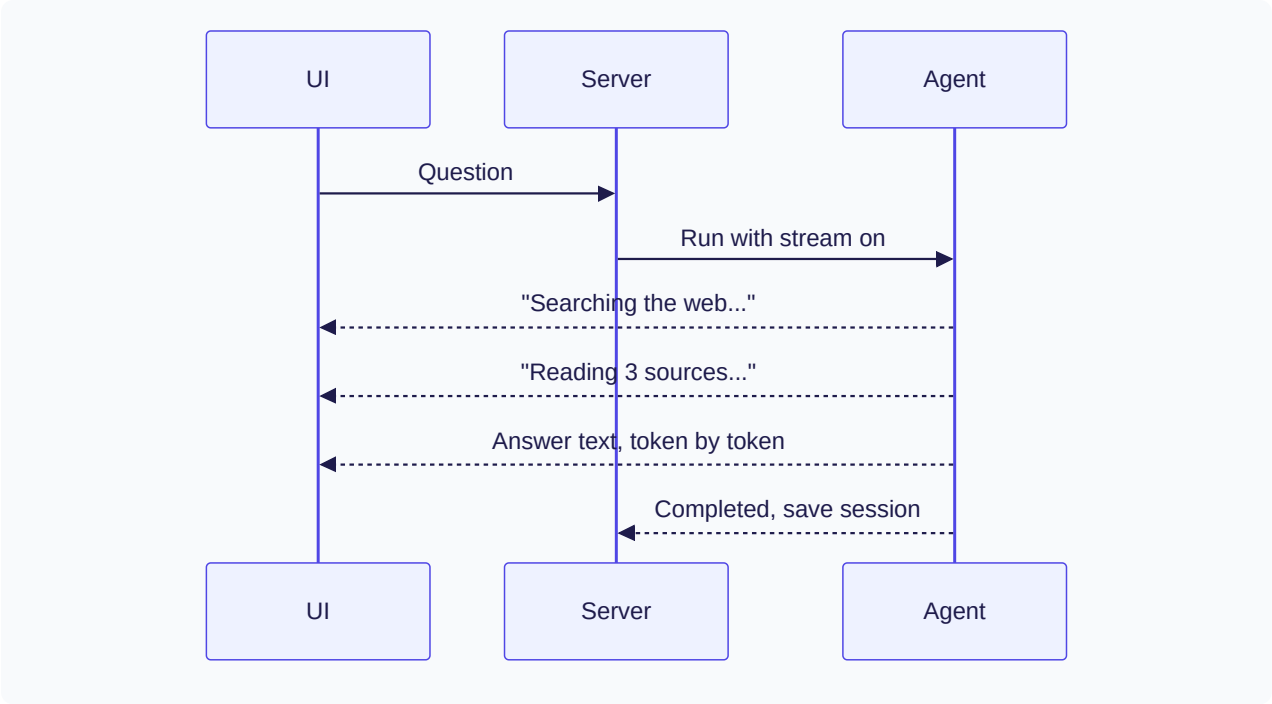
How to apply. Keep keys in env variables, never in prompts. Redact phone numbers, Aadhaar-like IDs and emails before logging. Get consent for memory, and let users delete their data.

Part 8 — Production

This is the difference between working on your laptop and something a stranger will tolerate using.

8.1 Streaming agent events

Real example. Perplexity shows "Searching… Reading 8 sources… Writing answer" as it works. The total time is similar, but users wait patiently because they see progress. A blank spinner for 20 seconds feels broken.

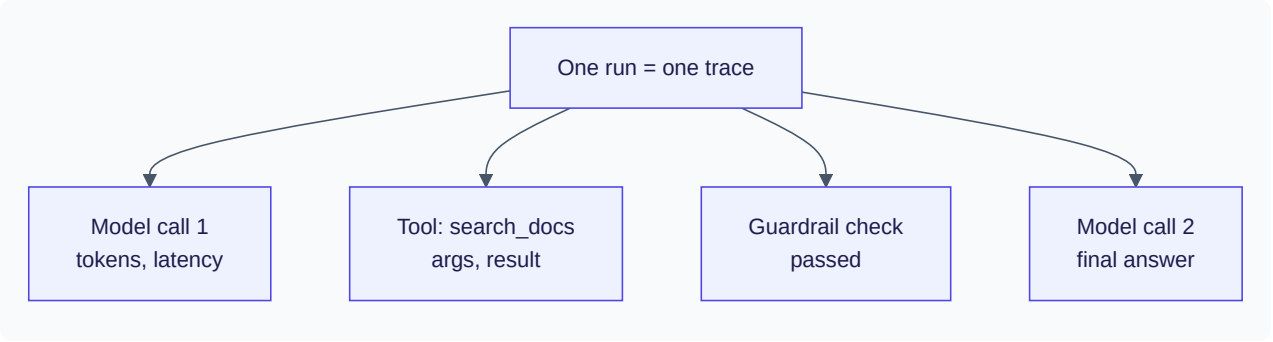


Where it applies. Every chat UI, research agents, coding agents, report generators.

How to apply. Stream from a Next.js route handler (Node runtime). Map tool-start events to status lines and stream text as it arrives. Wait for the run to fully complete before saving the session.

8.2 Tracing

Real example. A user says "your bot gave me a wrong answer yesterday." Without traces, you guess. With Langfuse or LangSmith, you open that exact run and see it picked the wrong tool on turn 2 because two descriptions overlapped.

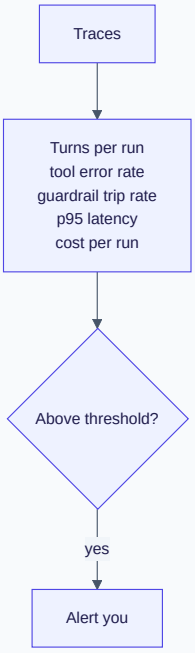


Where it applies. Every agent in production, and during development from day one.

How to apply. Add Langfuse (open source) or LangSmith. Tag each trace with user id, feature and prompt version. Open a trace after every surprising run. On serverless, flush traces before the function ends.

8.3 Metrics and alerts

Real example. If average turns per run rises from 3 to 7 after a prompt change, the agent is confused. The dashboard shows it days before users complain.

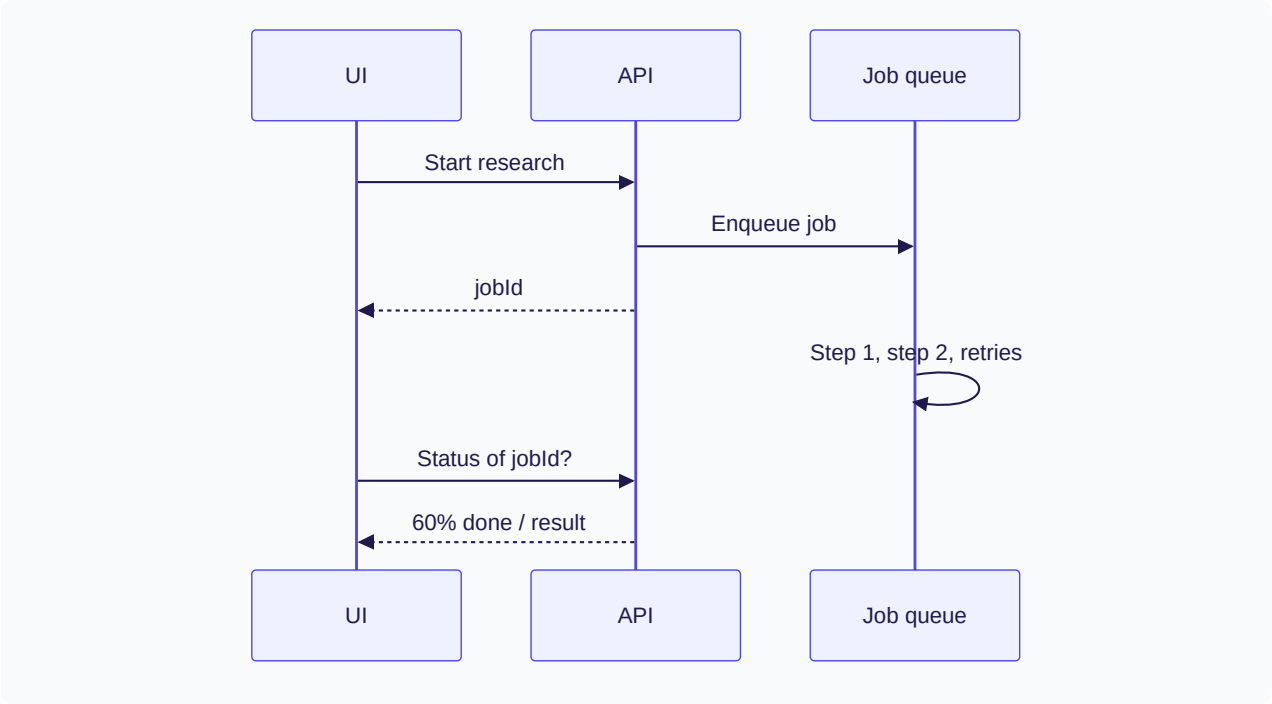


Where it applies. Any AI feature with real users.

How to apply. Log these five numbers per run. Alert on turns per run and cost per run first, because they move earliest.

8.4 Serverless limits and background jobs

Real example. Vercel functions have a maximum run time. A 5-minute research agent gets cut off mid-way. It needs a background job that the UI follows by job id.



Where it applies. Research agents, bulk document processing, video and audio pipelines, scheduled agents, approval flows.

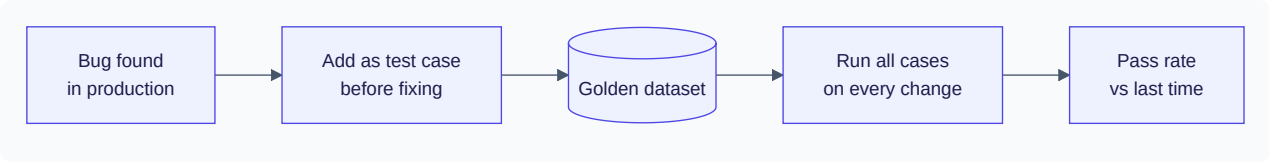
How to apply. Keep chat replies in the route handler. Move anything over about a minute to Trigger.dev or Inngest, and show progress by subscribing to the job.

Part 9 — Evals and cost

Evals are to AI features what unit tests are to code. Without them, every prompt change is a guess.

9.1 Golden datasets

Real example. Hamel Husain has written about his work on Rechat, a real-estate AI assistant, where progress stalled until the team built evals from real user failures. After that, every change could be measured.

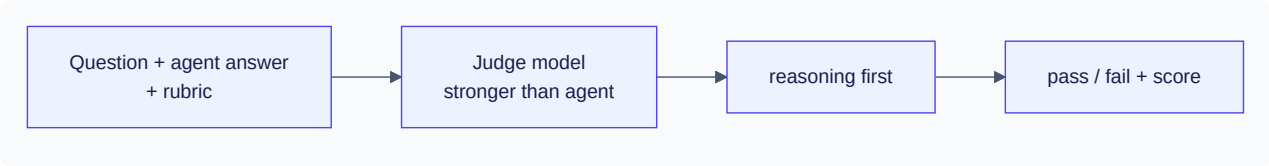


Where it applies. Every AI feature. Especially useful for multilingual input, spelling variants and edge cases (Hindi aliases, Hinglish, typos).

How to apply. Start with 20 cases, sourced in this order: past bugs, real user inputs, attack inputs, happy paths. Assert properties (which tool was called, which wasn't, number of turns), never exact text.

9.2 LLM-as-judge

Real example. You can't string-match "was this reply polite and accurate?" So a stronger model grades each answer against a written rubric.

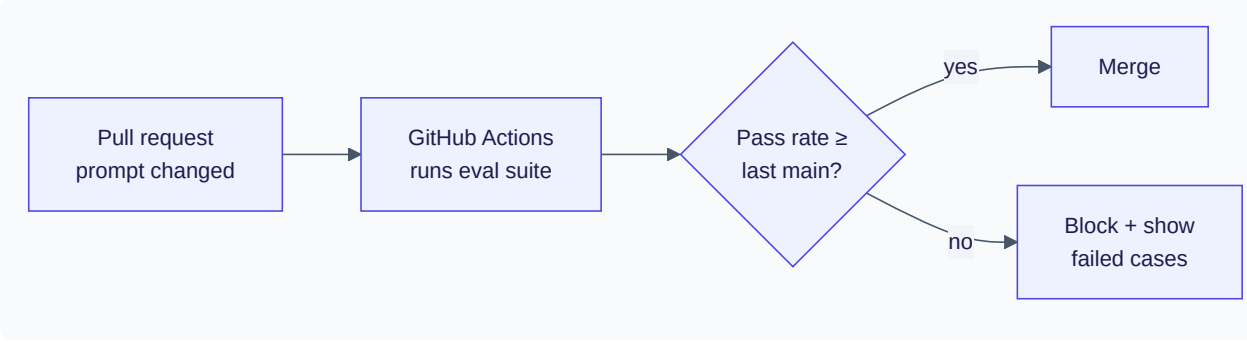


Where it applies. Tone, helpfulness, faithfulness to sources, summaries, translations, tutoring answers.

How to apply. Write a strict rubric. Put the reasoning field before the verdict in the output schema. Use a stronger model than the one being judged, and spot-check the judge by hand.

9.3 Evals in CI

Real example. A prompt tweak that improves Hindi answers silently breaks English refunds. A CI eval catches it before merge.

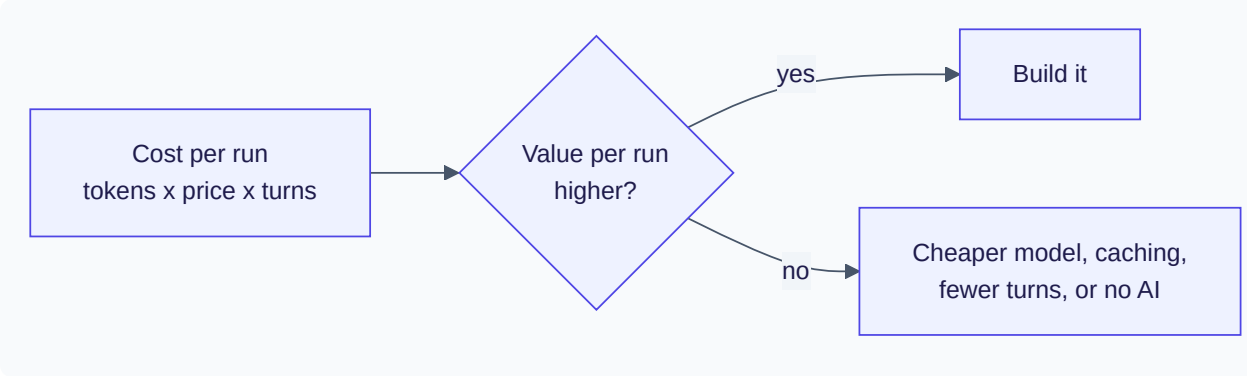


Where it applies. Any project where prompts, tools or models change regularly.

How to apply. Run the eval script in GitHub Actions on each pull request. Gate on "no regression" rather than 100%, and run each case 2–3 times because output varies.

9.4 Cost per run vs value per run

Real example. A support agent costs ₹4 per chat and replaces ₹40 of human time: a product. A recipe suggester costs ₹4 per chat and earns nothing: an expensive demo.



Where it applies. Every AI feature, before building and after launch.

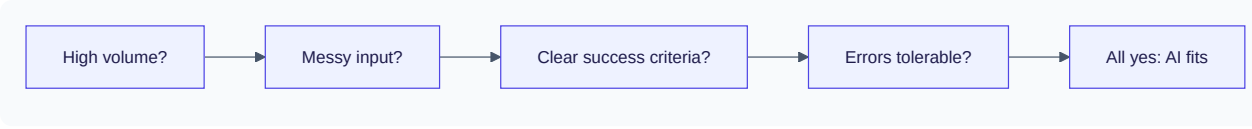
How to apply. Log tokens and cost per run with a user tag. Every turn re-sends the whole history, so long chats cost more than you'd expect. Levers: prompt caching, fewer tools per agent, cheap model for triage, compaction, stopping after the tool result.

Part 10 — Judgement and UX

The most valuable thing an agent engineer can say is "this doesn't need an agent." Users forgive mistakes they can see and fix; they don't forgive silent ones.

10.1 When AI earns its cost

Real example. Klarna announced in 2024 that its AI assistant was handling the work of hundreds of support staff: high volume, messy input, tolerable errors. By 2025 its CEO said publicly they'd put more emphasis on human support again for quality. Both halves are the lesson.

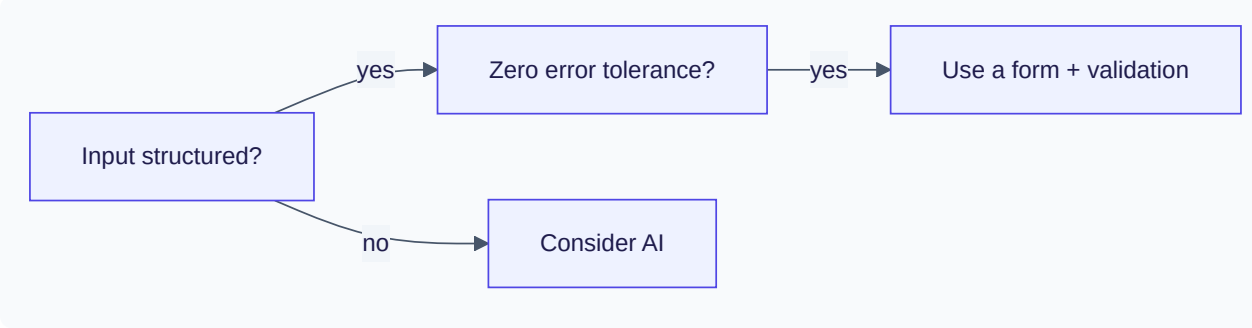


Where it applies. Support triage, document intake, classification, search, drafting.

How to apply. Check all four conditions. If any one fails, keep a human in the loop or reconsider AI.

10.2 When a form wins

Real example. IRCTC ticket booking has structured input and zero error tolerance. Dropdowns and validation beat a chatbot that might book the wrong date.



Where it applies. Payments, bookings, KYC, exam registration, government forms.

How to apply. Before building, ask: what is the cheapest thing that would work, why isn't it enough, and what does a wrong answer cost? AI can still help around the form, for example by pre-filling fields from a photo for the user to confirm.

10.3 Undo and visibility

Real example. Gmail's Undo Send and Claude Code's diff preview before applying edits work on the same principle: show the action, allow reversal.

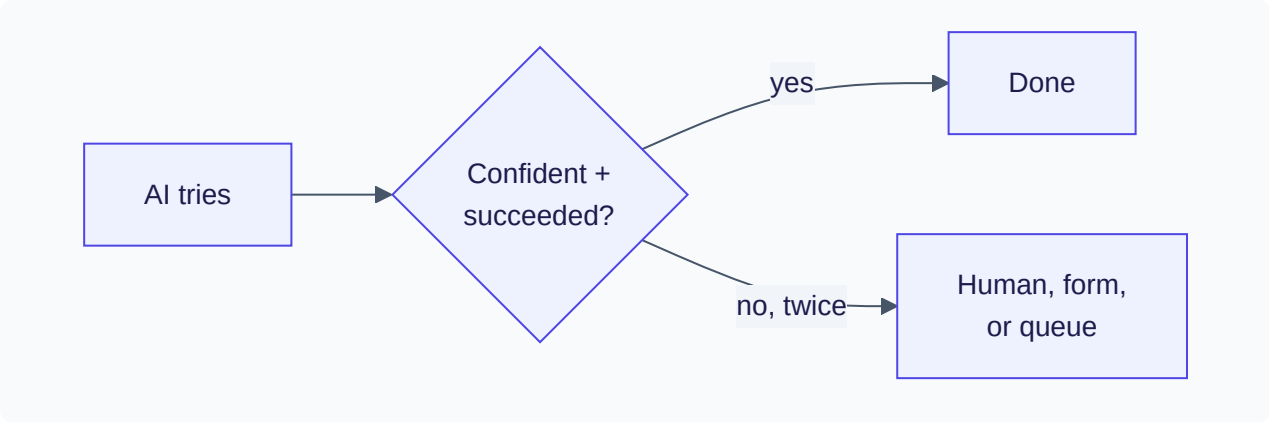


Where it applies. Writing tools, code editors, bulk data edits, AI-drafted emails, catalogue imports.

How to apply. AI drafts, human applies. Show sources and confidence, make output editable, and keep an undo for every AI write.

10.4 Fallbacks

Real example. When a bank's voice bot fails to understand you twice, it transfers you to a human. Every AI path needs a non-AI exit.



Where it applies. Voice bots, support chat, document extraction, any AI step on a critical path.

How to apply. Define the fallback before launch: a human handoff, a manual form, or a review queue. Trigger it on low confidence, repeated failure, or provider outage.

10.5 The eight gates

Run these in order for every feature, and stop at the first gate that rules a design out.

Gate	Question	Real example of the answer
1	Can rules solve it?	GST calculation: yes, so no AI
2	Is the path known in advance?	Loan workflow: known, so a workflow
3	What is the blast radius?	Refund: irreversible, so human approval
4	Where does state live?	Research job: in the DB, so it can resume
5	What is the minimum context?	Top 5 policy chunks, not the whole manual
6	How will I know it was wrong?	Traces + eval set before launch
7	What is the fallback?	Voice bot hands to a human
8	What does one run cost?	₹4 per chat vs ₹40 of saved time

Project types map

Every project type reuses the same topics in a different mix. Use this table to see which sections matter most for what you're building, and which real product to study.

Project type	Real product to study	Main flow	Key topics (sections)
Document Q&A	Notion AI, NotebookLM	Retrieve → answer with citations	2.1, 5.1–5.3, 9.2
Research agent	Perplexity, deep research features	Plan → parallel search → merge → report	3.1, 4.4, 8.1, 8.4
Coding agent	Claude Code, Cursor	Read repo → edit → run tests → approve	6.3, 7.4, 7.5, 5.5
Data / text-to-SQL	Uber's QueryGPT	Question → schema lookup → SQL → validate → run read-only	3.4, 3.5, 4.5, 9.1
Support agent	Intercom Fin, Klarna assistant	Triage → specialist → human fallback	4.6, 7.1, 7.2, 10.4
Browser agent	Claude for Chrome, OpenAI Operator	Read page → act → verify	7.3, 7.5, 6.3
Voice agent	Bank and airline phone bots	Speech → agent → speech, with interruptions	8.1, 10.4, 2.3
Workflow automation	Zapier AI, n8n	Trigger → fixed steps → one AI step	4.1, 4.2, 1.2, 1.3
Vision / extraction	Invoice and ID scanners	Photo → structured JSON → review queue	3.4, 1.4, 10.3
Personal assistant	ChatGPT / Claude memory	Chat → memory layers → tools	5.4, 7.6, 3.5
Content generation	Marketing and writing tools	Draft → critic → revise	4.5, 9.2, 1.4

One habit ties it all together. Whenever you use any AI product, run a quick teardown: what job it does, which pattern it uses, which tools it has, where it asks permission, where it feels slow, and what one change you'd make.